L'Architecture Client-Serveur et le Protocole HTTP

Thème principal : Architecture Web, protocole HTTP, méthodes GET/POST et codes de statut

Synthèse exécutive : Ce cours magistral pose les fondements indispensables du fonctionnement du Web en détaillant l'architecture client-serveur asymétrique et le protocole HTTP. L'étudiant y apprend comment un navigateur formule des requêtes textuelles standardisées, comment le serveur y répond de manière asynchrone et 'stateless' (sans état), et comment analyser les codes de statut HTTP (2xx, 3xx, 4xx, 5xx) ainsi que l'utilisation des méthodes GET et POST pour déboguer et concevoir des applications web sécurisées.

1. L'Architecture Client-Serveur et le Principe Stateless

L'architecture client-serveur est le modèle de communication standard du Web. Contrairement aux réseaux pair-à-pair (P2P) où chaque nœud est équivalent, les rôles sont ici strictement asymétriques : le client demande, le serveur exécute et répond. Une caractéristique majeure du protocole HTTP est d'être 'stateless' (sans état) : le serveur oublie instantanément le client une fois la réponse transmise. Bien que cette conception simplifie grandement la scalabilité et allège la charge des serveurs, elle pose un défi pour les applications modernes nécessitant un suivi de l'utilisateur (comme un panier d'achat ou une session connectée), ce qui oblige à recourir à des mécanismes additionnels comme les cookies et les sessions.

- Client actif : Logiciel (navigateur comme Firefox ou Chrome) qui initie la communication en envoyant une requête HTTP.
- Serveur Web passif : Processus logiciel distant (Apache, Nginx, IIS) qui écoute sur un port spécifique (80 pour HTTP, 443 pour HTTPS) et attend les requêtes.
- Protocole Stateless (Sans état) : Caractéristique fondamentale où chaque échange requête-réponse est totalement indépendant, sans mémoire des interactions passées.
- Mécanismes de contournement : Utilisation de cookies et de sessions pour maintenir une connexion utilisateur ou un panier d'achat.

Cas concret enseignant : *Le professeur compare HTTP à un guichetier amnésique : chaque fois que vous vous présentez devant lui, même si c'était il y a deux secondes, il vous traite comme un parfait inconnu. Pour qu'il se souvienne de vous, vous devez lui présenter un badge (le cookie de session) à chaque phrase. Il ajoute également que lors d'une simple connexion au site de l'IUT de Bayonne, le navigateur n'envoie pas une seule requête unique, mais des dizaines de requêtes simultanées pour récupérer le HTML, les images, les feuilles de style CSS et les scripts JS.*

2. Structure des Messages HTTP et Méthodes (GET vs POST)

Un message HTTP est purement textuel et structuré de manière rigoureuse. La requête comprend une ligne de requête (méthode, URI, version), des en-têtes (headers) fournissant des métadonnées essentielles comme le domaine cible via 'Host' ou le type de client via 'User-Agent', une ligne vide obligatoire, et un corps (body) optionnel. La réponse du serveur adopte une structure similaire mais commence par une ligne de statut. Le choix de la méthode est primordial : GET transmet les données dans l'URL avec des limites de taille (environ 2048 caractères), tandis que POST encapsule les données dans le corps de la requête. Attention, POST n'est pas sécurisé par défaut sans chiffrement HTTPS.

- Ligne de requête / statut : Contient la méthode, l'URI et la version du protocole pour la requête, ou le code de statut et sa description pour la réponse.
- En-têtes (Headers) : Métadonnées sous forme de paires clé-valeur (Host, User-Agent, Content-Type) décrivant le contexte.
- Méthode GET : Conçue pour la consultation de ressources (paramètres dans l'URL, sûre et idempotente).
- Méthode POST : Conçue pour modifier l'état du serveur (paramètres encapsulés dans le corps de la requête).

Cas concret enseignant : *Le professeur insiste sur le fait que le protocole HTTP est purement textuel et en clair par défaut. Utiliser la méthode GET pour transmettre un mot de passe constitue une faille de sécurité majeure (connue sous le nom de 'shoulder surfing'), car les données apparaissent en clair dans l'historique du navigateur ou à la vue d'une personne se trouvant derrière l'utilisateur.*

3. La Classification des Codes de Statut HTTP

Chaque réponse HTTP commence par un code numérique à trois chiffres permettant de classifier immédiatement le résultat. Il est crucial pour un développeur de distinguer les erreurs de la famille 4xx, qui incombent au client (mauvaise URL, syntaxe incorrecte), des erreurs de la famille 5xx, qui révèlent un dysfonctionnement interne de l'infrastructure ou du code serveur (script planté, base de données inaccessible), nécessitant une intervention technique directe.

- Codes 2xx (Succès) : La requête a été reçue et traitée avec succès (ex: 200 OK).
- Codes 3xx (Redirection) : Actions supplémentaires requises par le client (ex: 301 Moved Permanently).
- Codes 4xx (Erreur Client) : Anomalie dans la requête du client, URL erronée ou absence d'autorisation (ex: 404 Not Found, 403 Forbidden).
- Codes 5xx (Erreur Serveur) : Défaillance interne du serveur malgré une requête valide (ex: 500 Internal Server Error).

Cas concret enseignant : *Le professeur raconte qu'à ses débuts, il avait configuré par erreur une base de données qui plantait à chaque soumission de formulaire, renvoyant systématiquement une erreur 500. Les utilisateurs paniquaient en pensant que le site était piraté alors qu'il s'agissait simplement d'un point-virgule manquant dans son script PHP.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Monsieur, quelle est la différence concrète entre GET et POST pour envoyer un formulaire de connexion ?
Réponse du professeur : Avec GET, l'identifiant et le mot de passe sont ajoutés directement dans l'URL (ex: login.php?user=matei&pass;=1234), ce qui les rend visibles à l'écran, enregistrés dans l'historique du navigateur et stockés dans les logs du serveur. Avec POST, les identifiants sont placés dans le corps (body) de la requête HTTP, hors de vue dans l'URL. Cependant, attention : POST à lui seul ne chiffre pas les données, il faut obligatoirement utiliser HTTPS pour empêcher l'interception sur le réseau.

■ Exercice & Démonstration d'Application

Énoncé : Analyser la requête HTTP brute suivante et identifier la méthode utilisée, la ressource demandée, l'hôte cible, et expliquer ce que le serveur va renvoyer si la ressource n'existe pas : GET /images/logo.png HTTP/1.1 Host: iutbayonne.univ-pau.fr User-Agent: Mozilla/5.0

Résolution pas à pas :
1. Analyser la première ligne : 'GET /images/logo.png HTTP/1.1'. Le premier mot 'GET' est la méthode HTTP utilisée pour récupérer une ressource sans modifier l'état du serveur. L'URI cible est '/images/logo.png' et la version du protocole est 'HTTP/1.1'. 2. Identifier l'hôte cible en lisant l'en-tête 'Host: iutbayonne.univ-pau.fr', qui spécifie le domaine cible, indispensable pour le virtual hosting. 3. Analyser l'agent utilisateur : 'User-Agent: Mozilla/5.0' identifie le navigateur client. 4. Scénario d'absence de ressource : Si le fichier 'logo.png' n'existe pas dans le répertoire du serveur web, celui-ci ne peut pas honorer la demande et renvoie un code de statut de la classe 4xx, précisément le code '404 Not Found'.

Résultat : Méthode : GET, Ressource : '/images/logo.png', Hôte cible : 'iutbayonne.univ-pau.fr', Code de retour en cas d'absence : 404 Not Found (Erreur Client).

■ Auto-Évaluation Flash d'Examen

Q : Qu'est-ce qu'un protocole 'stateless' comme HTTP ?

R : C'est un protocole sans mémoire : le serveur traite chaque couple requête-réponse de manière complètement indépendante, sans conserver le souvenir des échanges passés avec le client.

Q : Quelle est la différence majeure entre les codes d'erreur de la série 4xx et ceux de la série 5xx ?

R : Les erreurs 4xx indiquent un problème imputable au client (URL incorrecte, ressource inexistante ou droits insuffisants), tandis que les erreurs 5xx désignent une panne ou un bogue interne du serveur.

Q : Pourquoi n'utilise-t-on jamais la méthode GET pour transmettre un mot de passe ?

R : Parce que GET fait transiter les paramètres directement en clair dans l'URL, ce qui les rend visibles à l'écran (shoulder surfing) et les enregistre dans l'historique de navigation ainsi que dans les journaux (logs) du serveur.